

CORS 크로스 도메인

CORS(Cross-Origin Resource Sharing)란?

- 웹 브라우저가 다른 도메인의 리소스에 접근할 때 적용되는 보안 정책입니다.

예시:

- 내 사이트: <https://mysite.com>
- API 서버: <https://api.example.com>

이때 mysite.com에서 api.example.com으로 Ajax 요청을 보내려고 하면 CORS 정책이 작동합니다.

왜 필요한가요?

보안상의 이유로 브라우저는 기본적으로 다른 도메인 간의 요청을 차단합니다. 악성 사이트가 사용자 모르게 다른 사이트의 데이터를 가져가는 것을 방지하기 위해서입니다.

JWT(Json Web Token)

JWT(Json Web Token)는 웹에서 인증(로그인)과 정보 전달에 자주 쓰이는 토큰 형식입니다.

핵심 개념

- JSON 형식의 데이터를 안전하게 전달하기 위한 토큰
- 보통 로그인 후 서버가 사용자에게 발급하고, 이후 요청마다 이 토큰을 보내서 인증을 확인
- 서버는 토큰 안에 들어 있는 서명(signature)을 검증해서 위조되지 않았음을 확인

장점

- 세션 저장소 없이 인증 가능 (stateless)
- JSON 기반이라 가볍고 다양한 시스템에서 사용 가능
- 클라이언트-서버 간 정보 전달에 편리

단점

- 페이로드는 암호화가 아닌 단순 인코딩이므로 민감한 정보 저장 금지
- 토큰이 유효한 동안은 탈취되면 위험
- 토큰을 강제로 무효화하는 게 어렵다는 점(만료 시간 기다려야 함)

Django REST Framework

- Django 웹 프레임워크 위에서 RESTful API를 쉽게 만들 수 있도록 도와주는 라이브러리에요.
- 웹 페이지 대신, JSON 형태의 데이터(API)를 주고받는 서버를 만들 때 주로 사용합니다.

프로젝트 생성

단계1: 모듈 추가 설치

```
pip install django
pip install djangorestframework
pip install django-cors-headers
pip install PyJWT
```

단계2: 프로젝트 & 앱 생성

```
django-admin startproject config .
python manage.py startapp api
python manage.py startapp user
python manage.py startapp authentication
```

단계3: settings.py

- 앱 추가

```
INSTALLED_APPS = [  
    ...,  
    "rest_framework",  
    "corsheaders",  
    "api",  
    "user",  
    "authentication"  
]
```

- User Model 설정

```
AUTH_USER_MODEL = "user.User"
```

- CORS 크로스 도메인 설정

```
MIDDLEWARE = [  
    "corsheaders.middleware.CorsMiddleware", # 맨위에 추가!!  
    ...  
]  
  
CORS_ORIGIN_ALLOW_ALL = True  
CORS_ALLOW_CREDENTIALS = True
```

- Django REST Framework 설정

```
REST_FRAMEWORK = {  
    'EXCEPTION_HANDLER': 'authentication.exceptions.status_code_handler'  
}
```

단계4: User Models 생성

- user.models.py

```
from django.contrib.auth.models import AbstractUser
from django.db import models

# Create your models here.
class User(AbstractUser):
    name = models.CharField(max_length=255)
    ...
```

단계5: User Serializer 생성

- 생성: user.serializer.py

```
from rest_framework.serializers import ModelSerializer
from .models import User

# api통신때 user정보를 제공하기 위한 객체
class UserSerializer(ModelSerializer):
    class Meta:
        model = User
        ...
```

단계6: exceptions 처리

- 생성: authentication.exceptions.py

```
from rest_framework.views import exception_handler

def status_code_handler(exc, context):
    response = exception_handler(exc, context)

    if response is not None:
        response.status_code = 777
        ...
```

단계7: Json Web Token

- 생성: authentication.token.py

```
import jwt
import datetime
import enum

from rest_framework.exceptions import AuthenticationFailed

class JWT_KEY(enum.Enum):
    RANDOM_OF_ACCESS_KEY = (enum.auto(), 'access_secret', datetime.timedelta(seconds=120), 'HS256', '랜덤한 조합의 키')
    RANDOM_OF_REFRESH_KEY = (enum.auto(), 'refresh_secret', datetime.timedelta(days=2), 'HS256', '랜덤한 조합의 키')
    ...
```

단계8: api.views.py

```
from rest_framework.authentication import get_authorization_header
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.exceptions import AuthenticationFailed , APIException

from user.models import User
from user.serializer import UserSerializer
from authentication.token import create_access_token, create_refresh_token, decode_access_token, decode_refresh_token

# Create your views here.
class Register(APIView):
    def post(self, request):
        serializer = UserSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
```

단계9: api.urls.py

```
from django.urls import path
from .views import Register, LoginView, HelloWorldView, RefreshView, Logoutview

urlpatterns = [
    path('register/', Register.as_view(), name='register'),
    path('login/', LoginView.as_view(), name='login'),
    path('hello-world/', HelloWorldView.as_view(), name='hello-world'),
```

단계10: config.urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("api/", include("api.urls"))
]
```

실행

단계1: 마이그레이션(migration)

```
$ (.venv) python manage.py makemigrations
```

단계2: Database에 마이그레이션 정보 반영

```
$ (.venv) python manage.py migrate
```

단계3: server 실행 및 관리자 페이지 접속

```
$ (.venv) python manage.py runserver
```

테스트

단계1: register

- <http://127.0.0.1:8000/api/register/>

```
{  
  "name":"hello", "email":"hello@gmail.com", "password":"hello1234"  
}
```

Register

Register

OPTIONS

GET /api/register/

HTTP 777
Allow: POST, OPTIONS
Content-Type: application/json
Vary: Accept

```
{  
  "detail": "Method \"GET\" not allowed."  
}
```

Media type:

application/json

Content:

```
{  
  "name": "hello", "email": "hello@gmail.com", "password": "hello1234"  
}
```

POST

- 결과 확인: 비밀번호는 보이지 않음!!

Django REST framework

Register

Register

OPTIONS

POST /api/register/

HTTP 200 OK
Allow: POST, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "id": 1,
  "name": "hello",
  "email": "hello@gmail.com"
}
```

- database에서 신규 회원 데이터 확인
 - 비밀번호는 hashing 처리!!

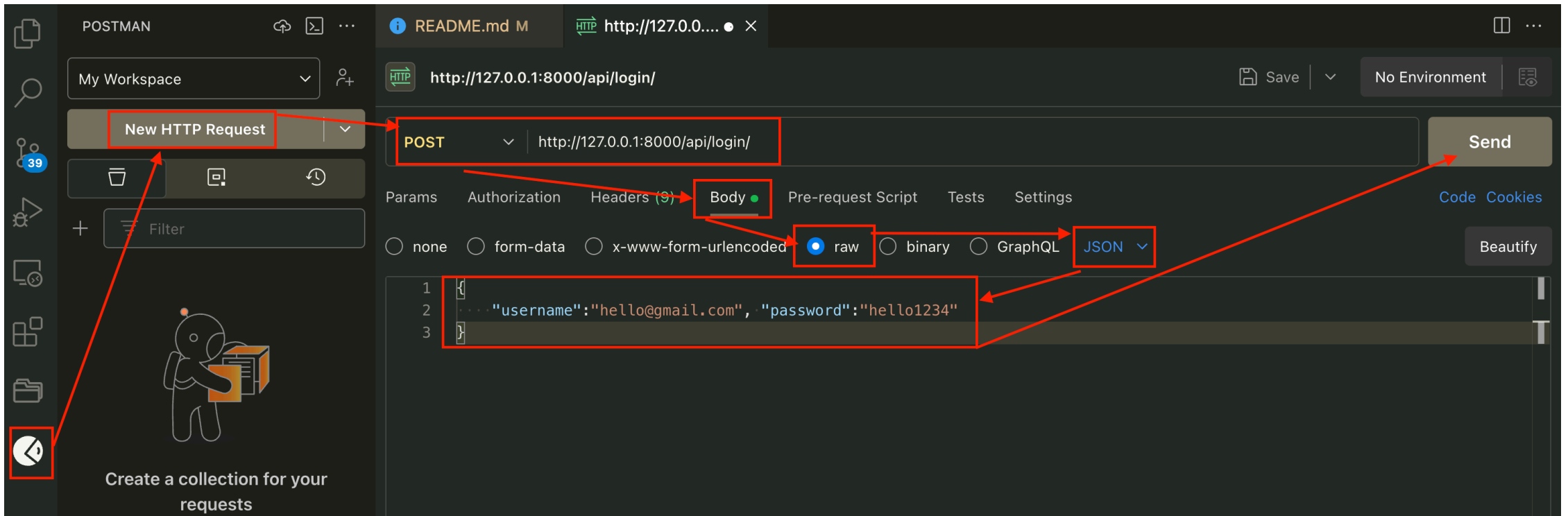
The screenshot shows a database explorer interface with a sidebar on the left and a main panel on the right. The sidebar, titled 'EXPLORER', contains a tree view under 'SQLITE EXPLORER' with the following items: db.sqlite3, auth_group, auth_group_permissions, auth_permission, django_admin_log, django_content_type, django_migrations, django_session, and user_user. The 'user_user' table is selected and highlighted with a red box. Below it, the table's schema is listed: id : integer, last_login : datetime, and is_superuser : bool. The main panel, titled 'SQLite', displays a table with the following columns: active, date_joined, name, email, and password. The table contains one row of data. A red arrow points from the 'user_user' table in the sidebar to the 'password' column in the table view.

active	date_joined	name	email	password
	2024-07-20 01:34:38.145379	hello	hello@gmail.com	pbkdf2_sha256\$600000\$3XfXMI0Qdj7tuLWy9G3MBB\$z6V43LxbEm8pNUhgtbJtx4HX+Eg9MOL5+UWAZRnHfn0=

단계2: login on postman

- <http://127.0.0.1:8000/api/login/>

```
{  
  "username": "hello@gmail.com", "password": "hello1234"  
}
```



- 결과 확인: access token 확인

The screenshot displays a REST client interface with a POST request to `http://127.0.0.1:8000/api/login/`. The request body is a JSON object containing `username` and `password`. The response status is `200 OK` with a time of `114 ms` and a size of `672 B`. The response body is a JSON object containing a `token`.

Request:

```
POST http://127.0.0.1:8000/api/login/

{
  "username": "hello@gmail.com", "password": "hello1234"
}
```

Response:

```
200 OK Time: 114 ms Size: 672 B

{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxCiJleHAiOjE3MjE0NTM3NzEsImhhdCI6MTcyMTQ1MzY1MX0.3hW0Dzqw-_432x4e9AhGNzGkxdNDQ-MPWUEiT4MdZXU"
}
```

- 결과 확인: refresh token 확인

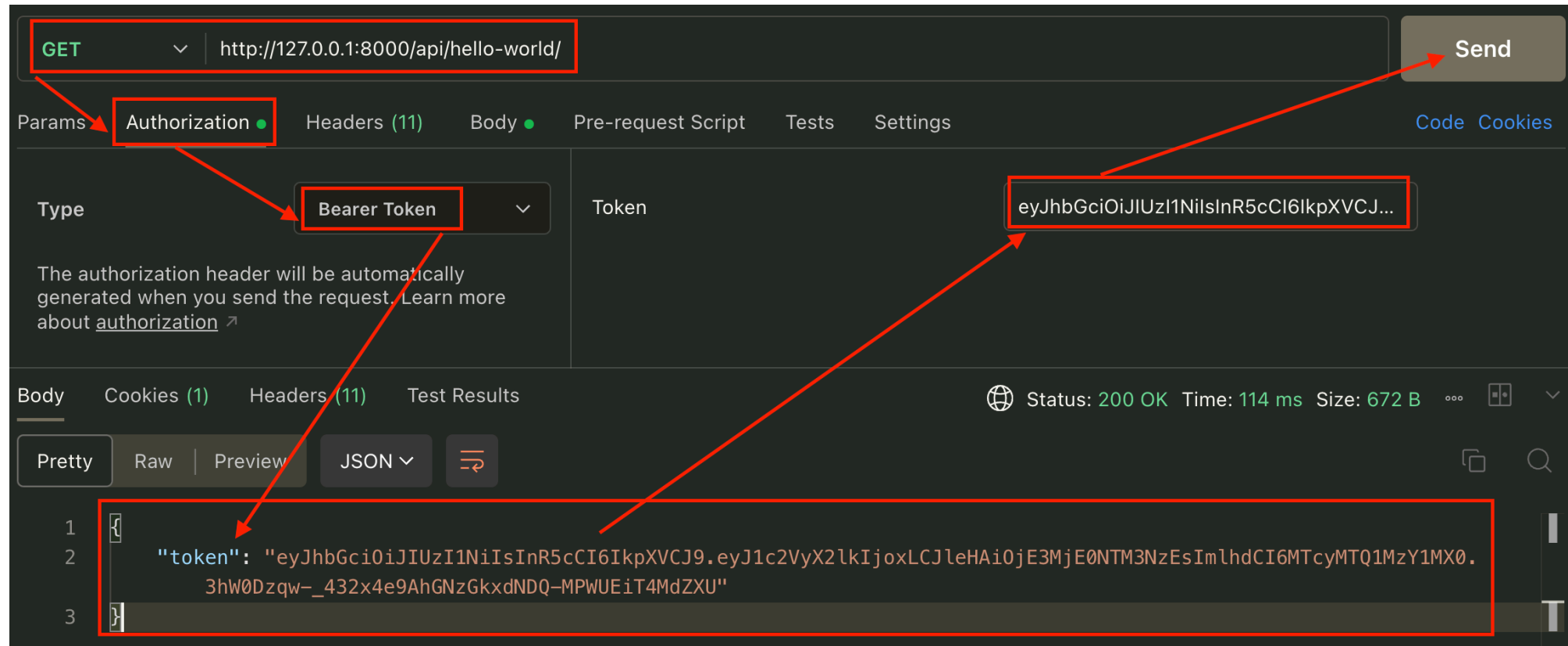
The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:8000/api/login/`. The request body is a JSON object: `{ "username": "hello@gmail.com", "password": "hello1234" }`. The response status is `200 OK` with a time of `114 ms` and size of `672 B`. The `Cookies (1)` tab is selected, showing a single cookie:

Name	Value	Domain	Path	Expires	HttpOnly	Secure
refreshToken	eyJhbGciOiJIUzI1Ni...	127.0.0.1	/	Session	true	false

단계3: hello world on postman

- <http://127.0.0.1:8000/api/hello-world/>

로그인 한 경우에만 접속 가능!!



- 결과 확인: 2분 이내로 호출해야함!!

The screenshot displays a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:8000/api/hello-world/
- Authorization:** Bearer Token (Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ...
- Status:** 200 OK
- Time:** 3 ms
- Size:** 356 B
- Response Body (JSON):**

```
{  "message": "Hello World"}
```

- 결과 확인: 2분이 지났기 때문에 access token이 만료됨

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:8000/api/hello-world/
- Authorization:** Bearer Token. The token value is eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ...
- Status:** 777 Unknown Status Code
- Time:** 3 ms
- Size:** 382 B
- Body:** {"detail": "Signature has expired"}

The response body is displayed in JSON format, showing an error message: {"detail": "Signature has expired"}.

단계4: refresh token을 이용한 access token 생성

- <http://127.0.0.1:8000/api/refresh/>

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://127.0.0.1:8000/api/refresh/
- Send Button:** Present in the top right.
- Tabs:** Params, Authorization, Headers (11), Body, Pre-request Script, Tests, Settings. The 'Body' tab is selected.
- Query Params:** A table with columns 'Key' and 'Value' is shown, but it is empty.
- Body Tab:** Contains a JSON response. The response is displayed in 'Pretty' format, with a 'JSON' dropdown menu and a 'Copy' icon.
- Status Bar:** Shows 'Status: 200 OK', 'Time: 4 ms', and 'Size: 483 B'.

The JSON response body is:

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJleHAiOjE3MjE0NTUyMzEsImhhdCI6MTcyMTQ1NTExMX0.PRoPi7oQ4m5Zegu6KJBT2o7PVe0y0QwAFQqlsplIfkk"
}
```

단계5: logout

- <http://127.0.0.1:8000/api/logout/>

logout을 하면 쿠키에 있는 refresh token 삭제됨

POST | http://127.0.0.1:8000/api/logout/ | Send

Params | Authorization | Headers (11) | Body | Pre-request Script | Tests | Settings | Code | Cookies

Query Params

Key	Value
-----	-------

Body | Cookies | Headers (11) | Test Results

Status: 200 OK | Time: 5 ms | Size: 434 B